AI ROI + Usage-Based Pricing Calculator

Hawkeye Go-To-Market Pack · 2026-04-26

AI ROI + Usage-Based Pricing Calculator

> *Track usage. Permission models. Tier model. ROI math. Calculator stub. Owner: founder + product · Last updated: 2026-04-26 · Status: spec + working calculator stub*

This doc defines:

1. **What we measure** — every agent invocation produces an `agent-usage-event`.
2. **Who can call what** — the permission matrix that lives in the Admin Panel.
3. **How we price it** — the tier model (free quota → metered → unlimited).
4. **What we charge for it** — the ROI calculator (time saved × hourly rate × usage count).
5. **How customers see their value** — the in-product ROI dashboard.

---

1. The 12 AI agents — current catalog + value

Agent	Module	What it does	Avg time saved per call	Token cost (cloud LLM)
1. Pre-Audit Questionnaire pre-fill	Audit · PREP	Pre-fills supplier questionnaire from supplier KB + ICH Q7 context	20-40 min/call	$0.02-0.05
2. Supplier-Intel	Audit · INITIATED + supplier pre-qual	Public-data fusion: openFDA + FDA WLs + EMA EudraGMDP + WHO PQ + verdict	30-60 min/call	$0.03-0.08
3. Audit Report Assembler	Audit · REPORTING	Drafts narrative report from findings + evidence + ICH Q7 framing	2-4 hours/call	$0.10-0.25
4. Observation Drafter (Wave-2)	Audit · EXECUTION	Suggests observation wording from cross-tenant anonymized findings	5-10 min/call	$0.02-0.04
5. CAPA RCA Drafter	CAPA	5-Whys / fishbone scaffold from finding text	15-30 min/call	$0.02-0.04
6. Risk Brainstormer	Risk Register	Generate top 8 risk scenarios (S/O/D × category)	20-30 min/call	$0.03-0.06
7. Complaint Triage	Complaint	Severity + MDR-reportability + recommended deadline	10-20 min/call	$0.02-0.04
8. Autofill Form (generic)	All	Fills any structured form from a reference doc	10-15 min/call	$0.02-0.04
9. Document KB Search (Ask Hawk)	All	RAG over tenant SOPs · regulatory citations · finds answer with source	15-30 min/call	$0.01-0.03
10. Deviation Triage	Deviation	Classifies deviation severity + FAR-reportability + initial RCA hypothesis	15-25 min/call	$0.02-0.04
11. Change Impact Analyzer	Change Control	Reads change description → maps blast radius (which docs, processes, products affected)	30-60 min/call	$0.04-0.08
12. Training Gap Analyzer	Training	Reads role definition + recent SOP changes → recommends training assignments	20-40 min/call	$0.02-0.05

Total time-saved opportunity (rough) for a 50-user pharma tenant doing 5,000 agent calls/month:

- Conservatively 15 min average × 5,000 = **1,250 hours/month saved**.
- At $40/hr loaded labor cost = **$50,000/month value generated**.
- Cloud LLM cost to deliver: ~$300/month.
- **ROI multiple: ~150x** at conservative assumptions.

(That ratio is the headline. Defending it is the rest of this doc.)

---

2. Usage event — the unit we measure

Every time an agent is invoked, we write one `agent-usage-event` row:

```
{
  _id: ObjectId,
  tenantId: 'acme-pharma-audit',         // who paid
  userId: ObjectId,                       // who clicked
  userRole: 'audit_lead',                 // for cohort analysis
  agentKey: 'audit.report.assemble',     // which agent
  agentVersion: '1.2.0',                  // prompt version
  provider: 'gemini-flash-lite',          // cloud LLM used
  inputTokens: 4231,                      // billable input
  outputTokens: 1820,                     // billable output
  totalTokens: 6051,
  costUsd: 0.0084,                        // computed cost
  durationMs: 3127,                       // wall-clock
  outcome: 'success',                     // success / blocked_by_quota / error / blocked_by_permi
  confidence: 0.86,                       // agent self-confidence
  groundedCitations: 4,                   // how many KB chunks cited
  linkedEntityType: 'audit',              // what record it acted on
  linkedEntityId: ObjectId,
  // ROI tracking
  estimatedTimeSavedMin: 180,             // calibrated per agent
  laborCostSavedUsd: 120.0,               // estimatedTimeSavedMin × tenant.laborRateUsd / 60
  // accepted? did the user keep the AI output?
  userAccepted: true,                     // did the user accept the AI output? (set on next user
  userEditedRatio: 0.18,                  // 0.0 = kept verbatim, 1.0 = rewrote everything
  createdAt: Date,
}
```

Indexes: `(tenantId, createdAt)`, `(tenantId, agentKey, createdAt)`, `(tenantId, userId, createdAt)`.

Retention: 24 months default; downsampled hourly aggregates after 90 days.

---

3. Permission matrix — who can call what

Lives in the Admin Panel (Doc 4 §2.6). Stored in `agent-permissions` collection.

```
{
  _id: ObjectId,
  tenantId: 'acme-pharma-audit',
  // Permission map: roleId → agentKey → policy
  permissions: {
    audit_lead: {
      'audit.report.assemble':   { allow: true,  dailyQuota: 50,   monthlyQuota: 1000 },
      'audit.observation.draft':  { allow: true,  dailyQuota: 200,  monthlyQuota: 4000 },
      'audit.preaudit.prefill':  { allow: true,  dailyQuota: 30,   monthlyQuota: 600  },
      'supplier.intel':          { allow: true,  dailyQuota: 50,   monthlyQuota: 800  },
      'capa.rca.draft':          { allow: true,  dailyQuota: 30,   monthlyQuota: 500  },
    },
    supplier_qa_head: {
      'audit.preaudit.prefill':  { allow: true,  dailyQuota: 10,   monthlyQuota: 100 },
      'capa.rca.draft':          { allow: true,  dailyQuota: 30,   monthlyQuota: 300 },
      'doc.askhawk.search':      { allow: true,  dailyQuota: 50,   monthlyQuota: 1000 },
    },
    buyer_purchase: {
      'supplier.intel':          { allow: true,  dailyQuota: 20,   monthlyQuota: 300 },
      'doc.askhawk.search':      { allow: true,  dailyQuota: 50,   monthlyQuota: 1000 },
      // everything else: deny
    },
    // ... other roles
  },
  // Tenant-level cap (across all users)
  tenantQuota: {
    monthlyTokenLimit: 10_000_000,        // hard ceiling
    monthlyCostLimitUsd: 500,             // hard $ ceiling
    enforcement: 'hard',                   // 'hard' / 'soft' / 'unlimited'
    alertAt: [0.7, 0.9, 1.0],             // notify tenant_admin
  },
  defaultPolicy: 'deny',                  // any role × agent combo not listed = deny
  updatedAt: Date,
  updatedBy: ObjectId,
}
```

Policy resolution order

When a user invokes an agent, the runtime resolves policy in this order:

1. **User-specific override** (rare, set by tenant_admin) — explicit allow/deny + per-user quota.
2. **Role policy** for the user's role.
3. **Tenant default policy** (deny by default).
4. **Tenant-level quota** (always enforced as final ceiling).

If any check fails → `outcome: 'blocked_by_permission'` or `'blocked_by_quota'`, `agent-usage-event` is still written (for analytics), but the LLM is NOT called.

---

4. Tier model — how we price AI

Three pricing tiers (separate from the per-vertical-pack pricing in Doc 1):

Tier A — Included (default with every plan)

- **Quota**: 5,000 agent calls / month / tenant for SaaS · 10,000 / month for Private Cloud · unlimited for On-prem (you bring your own LLM).
- **Throttling**: per-user soft caps (configurable in Admin Panel).
- **Models**: Gemini Flash-Lite (free tier).
- **Price impact**: $0 — bundled in engine + pack price.

Tier B — Metered (overage / premium models)

- **Overage on Tier A**: $0.005 per call beyond included quota (vs ~$0.005-0.02 actual cost — modest margin).
- **Premium models** (Claude Sonnet · GPT-4-class) — opt-in per agent: $0.05-0.10 per call (cost-pass-through with 30% margin).
- **PII Redaction Proxy** (hybrid deployments only): $0.002 per call surcharge.

Tier C — Unlimited (enterprise + on-prem)

- **Unlimited cloud LLM calls**, capped only at tenant-set $ ceiling.
- **Bring-your-own LLM key** option (customer pays Gemini/Anthropic directly, Hawkeye charges only platform fee + 10% management fee).
- **On-prem LLM** (no cloud LLM cost — customer's GPUs).
- **Price impact**: +$10k-30k/yr add-on.

Pricing summary

Tier	Inclusion	Overage	Premium models
A Included	5k calls / month (SaaS), 10k (Private Cloud)	n/a	n/a
B Metered	+ $0.005/call beyond	yes	+ $0.05-0.10/call
C Unlimited	unlimited	none	included

---

5. The ROI calculator (working stub)

A drop-in JS calculator that takes tenant config + actual usage from `agent-usage-events` and produces the ROI dashboard the customer sees.

```
// scripts/lib/roiCalculator.js
// Stub: pure function. Wire up to /api/admin/ai/roi handler.

/**
 * @param {Object} params
 * @param {Array<UsageEvent>} params.events  - 30 days of agent-usage-events
 * @param {Object} params.tenant             - tenant config: { laborRateUsd, currency }
 * @param {Object} params.agentCatalog       - per-agent: { estimatedTimeSavedMin, displayName }
 * @returns {Object} ROI report
 */
export function computeRoi({ events, tenant, agentCatalog }) {
  const laborRate = tenant.laborRateUsd ?? 40; // default $40/hr loaded
  const periodDays = 30;

  let totalCalls = 0;
  let totalCost = 0;
  let totalTimeSavedMin = 0;
  let totalLaborSaved = 0;
  let acceptedCalls = 0;
  let blockedCalls = 0;
  const perAgent = {};

  for (const e of events) {
    totalCalls++;
    totalCost += e.costUsd ?? 0;
    if (e.outcome === 'success') {
      const tMin = e.estimatedTimeSavedMin ?? agentCatalog[e.agentKey]?.estimatedTimeSavedMin ?? 0
      const acceptanceWeight = e.userAccepted ? 1 - (e.userEditedRatio ?? 0) * 0.5 : 0.3;
      const adjustedMin = tMin * acceptanceWeight;
      totalTimeSavedMin += adjustedMin;
      totalLaborSaved += (adjustedMin / 60) * laborRate;
      if (e.userAccepted) acceptedCalls++;
    } else if (e.outcome.startsWith('blocked_')) {
      blockedCalls++;
    }
    const k = e.agentKey;
    perAgent[k] = perAgent[k] ?? { calls: 0, cost: 0, timeSavedMin: 0, accepted: 0 };
    perAgent[k].calls++;
    perAgent[k].cost += e.costUsd ?? 0;
    perAgent[k].timeSavedMin += e.estimatedTimeSavedMin ?? 0;
    if (e.userAccepted) perAgent[k].accepted++;
  }

  const roiMultiple = totalCost > 0 ? totalLaborSaved / totalCost : Infinity;
  const acceptanceRate = totalCalls ? acceptedCalls / totalCalls : 0;

  return {
    period: { days: periodDays, startedAt: events[events.length - 1]?.createdAt, endedAt: events[0
    headline: {
      totalCalls,
      totalTimeSavedHours: Math.round(totalTimeSavedMin / 60),
      totalLaborSavedUsd: Math.round(totalLaborSaved),
      totalCostUsd: Math.round(totalCost * 100) / 100,
      roiMultiple: Math.round(roiMultiple * 10) / 10,    // e.g., 150.4x
```

```
      acceptanceRate: Math.round(acceptanceRate * 100), // e.g., 84%
      blockedCalls,
    },
    perAgent: Object.entries(perAgent).map(([key, v]) => ({
      agentKey: key,
      displayName: agentCatalog[key]?.displayName ?? key,
      calls: v.calls,
      costUsd: Math.round(v.cost * 100) / 100,
      timeSavedHours: Math.round(v.timeSavedMin / 60),
      laborSavedUsd: Math.round((v.timeSavedMin / 60) * laborRate),
      acceptanceRate: v.calls ? Math.round((v.accepted / v.calls) * 100) : 0,
    })).sort((a, b) => b.laborSavedUsd - a.laborSavedUsd),
    projection: {
      monthlyTimeSavedHours: Math.round((totalTimeSavedMin / 60) * (30 / periodDays)),
      monthlyLaborSavedUsd: Math.round(totalLaborSaved * (30 / periodDays)),
      annualLaborSavedUsd: Math.round(totalLaborSaved * (365 / periodDays)),
    },
  };
}
```

What the customer sees (Admin Panel · Section 2.6 · "ROI Dashboard")

```
┌──────────────────────────────────────────────────────────────────┐
│  AI Agent ROI · last 30 days                                     │
├──────────────────────┬──────────────────────┬────────────────────┤
│   1,247 hours saved  │   $49,880 labor saved  │   $312 LLM cost  │
├──────────────────────┼──────────────────────┼────────────────────┤
│   ROI multiple: 159.9x │   Acceptance rate: 87%  │   Blocked: 3 calls  │
└──────────────────────┴──────────────────────┴────────────────────┘

Top agents this period:
  1. Audit Report Assembler          43 calls · 172 hrs · $6,880 saved · 91% acc
  2. Pre-Audit Questionnaire prefill 218 calls · 145 hrs · $5,800 saved · 88% acc
  3. Supplier-Intel                  61 calls · 91 hrs · $3,640 saved · 79% acc
  ...
Projected annual: 15,164 hrs · $607k labor saved · $3.8k LLM cost
```

This is the **single most powerful screen in the product for renewals**. Customer asks the CFO to renew; CFO sees `$607k annual labor saved` vs `$50k annual contract`; renewal closes.

---

6. Implementation plan

Phase 1 — Instrument every agent (Q3 2026)

Every existing agent endpoint already calls `groundedGenerate.js`. Wrap that call with a usage-event writer:

```
// In src/services/ai/runtime/groundedGenerate.js (or new wrapper)
export async function groundedGenerateInstrumented(params, context) {
  const start = Date.now();
  const { tenantId, userId, userRole, agentKey, linkedEntityType, linkedEntityId } = context;

  // ── Permission check ──
  const policy = await resolveAgentPolicy({ tenantId, userId, userRole, agentKey });
  if (!policy.allow) {
    await writeUsageEvent({ ...context, outcome: 'blocked_by_permission', durationMs: 0 });
    throw new HttpError(403, 'agent_permission_denied', { agentKey, policy });
  }

  // ── Quota check ──
  const quota = await checkQuota({ tenantId, userId, agentKey });
  if (quota.exhausted && quota.enforcement === 'hard') {
    await writeUsageEvent({ ...context, outcome: 'blocked_by_quota', durationMs: 0 });
    throw new HttpError(429, 'agent_quota_exhausted', { quota });
  }

  // ── Call LLM ──
  const result = await groundedGenerate(params);

  // ── Write usage event ──
  const event = {
    tenantId, userId, userRole, agentKey,
    agentVersion: params.promptVersion,
    provider: result.provider,
    inputTokens: result.inputTokens,
    outputTokens: result.outputTokens,
    totalTokens: result.totalTokens,
    costUsd: computeCost(result.provider, result.inputTokens, result.outputTokens),
    durationMs: Date.now() - start,
    outcome: result.ok ? 'success' : 'error',
    confidence: result.confidence,
    groundedCitations: result.citations?.length ?? 0,
    linkedEntityType, linkedEntityId,
    estimatedTimeSavedMin: AGENT_CATALOG[agentKey]?.estimatedTimeSavedMin ?? 0,
    laborCostSavedUsd: 0, // computed at read-time using tenant.laborRateUsd
    userAccepted: null,    // backfilled when user accepts/rejects
    userEditedRatio: null, // backfilled
    createdAt: new Date(),
  };
  await writeUsageEvent(event);
  return result;
}
```

Phase 2 — Backfill `userAccepted` (Q4 2026)

Each agent's UI gets two buttons: `Accept` and `Discard`. On click, the frontend POSTs the acceptance + (for accept) the edit-ratio computed by diffing AI output vs final saved value. Backfills the matching `agent-usage-event`.

Phase 3 — Admin Panel UI (Q4 2026)

Build the AI Agents section (Doc 4 §2.6) on top of `agent-permissions` and `agent-usage-events`:

- Permissions matrix editor.
- Quota dial.
- ROI dashboard (the screen above).
- Cost projection.

Phase 4 — Customer-facing API (Q1 2027)

`GET /api/admin/ai/roi?period=30d` returns the JSON the calculator emits — for tenants that want to pipe ROI into their own BI tools.

---

7. The ROI calculator goes in the deck

For sales conversations:

> *"On day 1 of the contract, we install the ROI dashboard. By month 3, you have hard evidence of ~$50k/month in labor savings vs ~$300/month in LLM cost. Bring that to your CFO when it's time to renew. We'll lose deals where the ROI doesn't materialize — and that's the right way to lose them."*

The calculator is:

- Customer-facing (in their Admin Panel).
- Sales-facing (in the deck, with anonymized real numbers from existing tenants).
- Renewal-facing (auto-emailed to tenant_admin monthly).
- Investor-facing (we report aggregated cohort ROI as a board metric).

---

8. The honest caveats

For your IT-pitch (Doc 6 Track B), be ready:

- **Time-saved estimates are calibrated, not measured.** We assume 20-40 min saved per pre-fill. Acceptance rate × edit ratio adjusts. Should we A/B test against unaided baseline? Yes — Phase 4.
- **Acceptance rate ≠ accuracy.** A user who accepts a wrong AI suggestion still produces wrong work. We're tracking `accuracy_at_review` separately starting Q1 2027 (downstream review flags).
- **LLM cost is variable.** Gemini Flash-Lite is free for now; Anthropic price changes will pass through to Tier B customers.
- **Quota is aggressive by default.** Tenant_admins can dial up; we'd rather block-and-warn than burn the customer's wallet.

---

9. Cross-references

- Permission matrix UI → see Doc 4 §2.6
- Tenant pricing tiers → see Doc 1 §7
- Cloud-vs-on-prem LLM choice → see Doc 3
- Two-track sales pitch (executive ROI + IT compliance) → Doc 6